🟢 Aula 2 — Template Syntax e Data Binding (4h)

🎯 Objetivos da aula

- Entender **como o Vue conecta dados ao HTML** (Template Syntax).
- Usar **interpolação**, **diretivas** e **two-way binding**.
- Construir um **formulário reativo**, com **renderização condicional** e **listagem dinâmica**.
- Consolidar o conceito de **reatividade** na prática.

---

1) Teoria (aprox. 1h30)

1.1 Interpolação (`{{ }}`)

- Forma mais simples de exibir dados do estado no template.
- Só funciona em **texto** (não em atributos HTML).

```
<p>Nome: {{ nome }}</p> <p>Total de itens: {{ itens.length }}</p>
```

✔️ Atualiza automaticamente quando o estado muda.

1.2 Diretivas do Vue

Diretivas são **atributos especiais** que começam com `v-` .

`v-bind`

- Liga atributos HTML a dados do Vue.
- Forma curta: `:atributo`

```
<img :src="imagemUrl" /> <button :disabled="isDisabled">Salvar</button>
```

`v-if`, `v-else-if`, `v-else`

- Renderização **condicional real** (remove do DOM).

```
<p v-if="logado">Usuário logado</p> <p v-else>Usuário visitante</p>
```

`v-for`

- Renderiza listas dinamicamente.
- **Sempre usar** `:key`.

```
<li v-for="item in lista" :key="item.id"> {{ item.nome }} </li>
```

1.3 Two-way binding (`v-model`)

- Conecta **input** ↔ **estado**.
- Atualiza o estado quando o usuário digita **e** a UI quando o estado muda.

```
<input v-model="email" /> <p>Email digitado: {{ email }}</p>
```

✔ Muito usado em formulários.

---

2) Prática (aprox. 2h30) — Projeto didático completo

🎯 **Contexto da prática**

Criar um **mini cadastro de usuários** com:

- **Formulário (v-model)**
- **Renderização condicional (v-if)**
- **Listagem dinâmica (v-for)**

2.1 Estrutura usada

Vamos trabalhar **apenas no** `App.vue` nesta aula.

2.2 `App.vue` — Código completo e comentado

Substitua o conteúdo do `src/App.vue` por este código:

```
<template> <main class="container"> <h1>ILP3 – Vue.js Aula 2</h1> <!-- ============================== --> <!-- FORMULÁRIO COM TWO-
WAY BINDING --> <!-- ============================== --> <section class="card"> <h2>Cadastro de Usuário</h2> <!-- v-model cria
ligação direta input <-> estado --> <input v-model="form.nome" type="text" placeholder="Nome" /> <input v-model="form.email"
type="email" placeholder="Email" /> <button @click="adicionarUsuario" :disabled="!form.nome || !form.email" > Adicionar </button>
<!-- Renderização condicional --> <p v-if="erro" class="error"> {{ erro }} </p> </section> <!-- ============================== -->
<!-- LISTAGEM DINÂMICA --> <!-- ============================== --> <section class="card"> <h2>Usuários Cadastrados</h2> <!--
Condição para lista vazia --> <p v-if="usuarios.length === 0"> Nenhum usuário cadastrado. </p> <!-- v-for para lista dinâmica -->
<ul v-else> <li v-for="usuario in usuarios" :key="usuario.id" > <strong>{{ usuario.nome }}</strong> – {{ usuario.email }} <!--
Exemplo de v-bind com classe dinâmica --> <span class="status" :class="{ ativo: usuario.ativo }" > {{ usuario.ativo ? 'Ativo' :
'Inativo' }} </span> </li> </ul> </section> <!-- ============================== --> <!-- CONTROLE CONDICIONAL --> <!--
============================== --> <section class="card"> <h2>Controle</h2> <button @click="alternarVisibilidade"> {{ mostrarLista
? 'Ocultar' : 'Mostrar' }} Lista </button> <!-- v-if controla renderização do bloco --> <div v-if="mostrarLista" class="info">
Total de usuários: {{ usuarios.length }} </div> </section> </main> </template> <script> export default { name: 'App', data() {
```

```
return { // Estado do formulário form: { nome: '', email: '' }, // Lista dinâmica de usuários usuarios: [], // Controle de erro erro: '', // Controle de renderização mostrarLista: true, // Simples gerador de ID proximoId: 1 } }, methods: { adicionarUsuario() { // Validação simples if (!this.form.nome || !this.form.email) { this.erro = 'Preencha todos os campos.' return } // Adiciona novo usuário na lista this.usuarios.push({ id: this.proximoId++, nome: this.form.nome, email: this.form.email, ativo: true }) // Limpa formulário e erro this.form.nome = '' this.form.email = '' this.erro = '' }, alternarVisibilidade() { this.mostrarLista = !this.mostrarLista } } } </script> <style scoped> .container { max-width: 900px; margin: 40px auto; font-family: Arial, sans-serif; } .card { border: 1px solid #ddd; padding: 20px; margin-bottom: 20px; border-radius: 10px; } input { display: block; margin-bottom: 10px; padding: 8px; width: 100%; } button { padding: 8px 14px; cursor: pointer; } .error { color: #c62828; margin-top: 10px; } ul { padding-left: 20px; } li { margin-bottom: 8px; } .status { margin-left: 10px; padding: 2px 6px; border-radius: 6px; font-size: 12px; background: #ccc; } .status.ativo { background: #4caf50; color: white; } .info { margin-top: 10px; font-weight: bold; } </style>
```

---

3) O que foi aplicado nesta aula (ligando teoria à prática)

✔ Interpolação

```
{{ usuario.nome }} {{ usuarios.length }}
```

✔ Diretivas

- `v-model` → formulário
- `v-if / v-else` → controle condicional
- `v-for` → listas
- `v-bind ( : )` → atributos e classes dinâmicas

✔️ Two-way binding

```
<input v-model="form.nome" />
```

---

4) Atividade prática da aula (avaliativa)

🎯 Desafio

1. Adicionar um botão "**Ativar/Inativar**" ao lado de cada usuário
2. O botão deve:
   - Alternar `usuario.ativo`
   - Atualizar o texto e a cor automaticamente
3. Exibir no topo:
   - "Usuários ativos: X"

💡 Dica:

```
<button @click="usuario.ativo = !usuario.ativo"> {{ usuario.ativo ? 'Inativar' : 'Ativar' }} </button>
```

---

5) Checklist de encerramento

- Usou interpolação (`{{ }}`)
- Aplicou `v-if` e `v-for`
- Criou formulário com `v-model`
- Observou reatividade sem refresh
- Entendeu o fluxo estado → UI